WIRESHARK REPORT

OVERVIEW OF PACKET CAPTURE FOR DETECTION OF MALICIOUS ATTACKS NETWORK ACTIVITY

Aim:

To simulate a cyberattack using a port scanning technique and analyzing the generated network traffic using Wireshark. Additionally, a Python-based automation approach is used to efficiently process and analyze the captured data.

Objective

- To simulate a port scanning attack using Nmap
- To capture and analyze network packets using Wireshark
- To identify suspicious patterns in network traffic
- To automate packet analysis using Python and Scapy

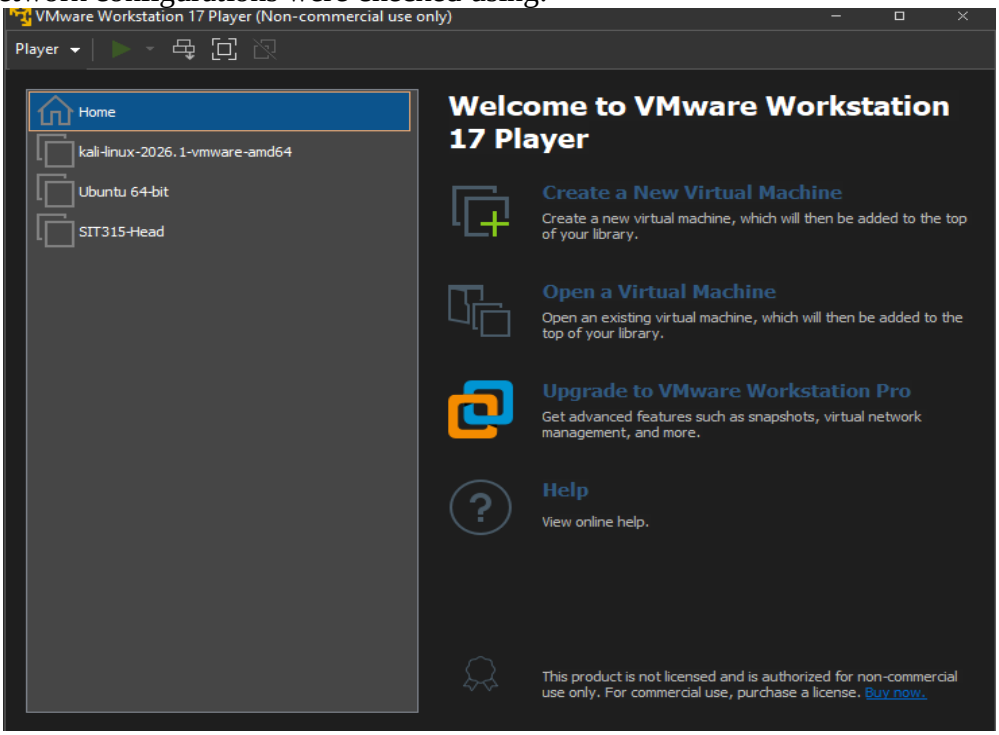
Tools and Environment

- **Kali Linux** – Used as the attacking machine
- **VMware Workstation 17 Player** – Virtualization platform
- **Windows Host System** – Target system
- **Wireshark** – Network packet analyzer
- **Python (Scapy Library)** – Automation and analysis

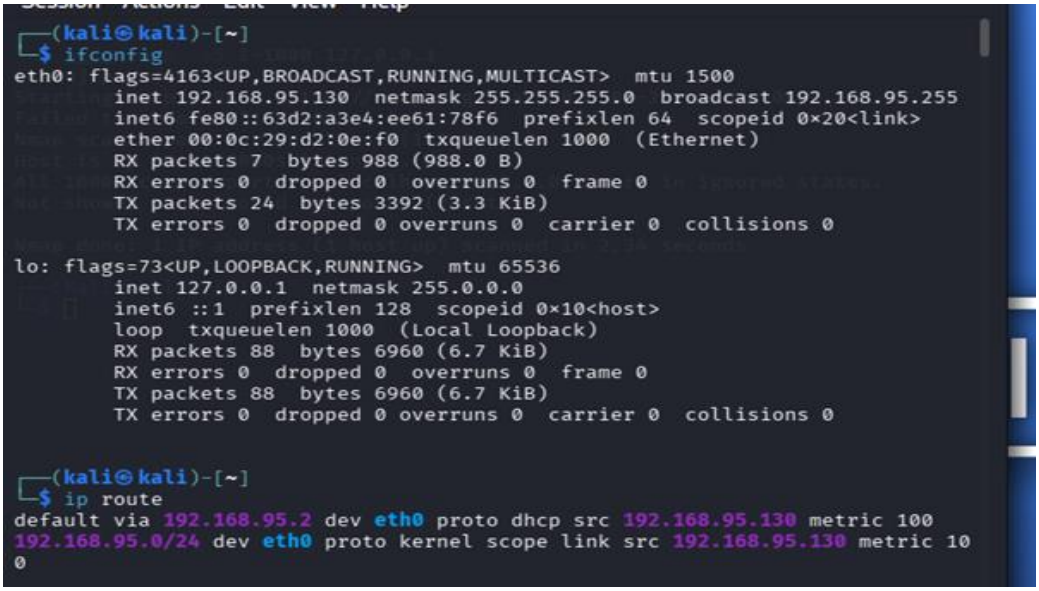
Methodology:

1. **Setup**

The Kali Linux virtual machine was launched using VMware Workstation Player. Network configurations were checked using:



Ifconfig – for linux system



Ipconfig – for host system (windows)

```
   Default Gateway . . . . . . . . . :

Unknown adapter Local Area Connection:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 2:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . : hgu_lan
   IPv4 Address. . . . . . . . . . . : 192.168.1.46
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.1.1

Ethernet adapter Bluetooth Network Connection:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

C:\Users\jaitl>
```

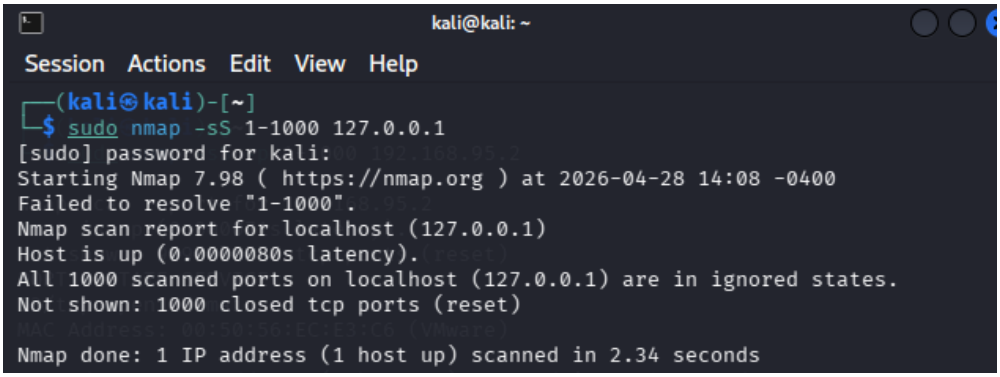
2. Packet Capture

Wireshark was launched on the Kali Linux system:

```
┌──(kali㉿kali)-[~]
└─$ sudo wireshark
 ** (wireshark:6996) 14:06:07.182707 [Capture MESSAGE] -- Capture Start ...
 ** (wireshark:6996) 14:06:07.211440 [Capture MESSAGE] -- Capture started
 ** (wireshark:6996) 14:06:07.211790 [Capture MESSAGE] -- File: "/tmp/wiresha
rk_eth0X76LO3.pcapng"
 ** (wireshark:6996) 14:09:16.376718 [Capture MESSAGE] -- Capture Stop ...
 ** (wireshark:6996) 14:09:16.425071 [Capture MESSAGE] -- Capture stopped.
 ** (wireshark:6996) 14:09:33.164207 [Capture MESSAGE] -- Capture Start ...
 ** (wireshark:6996) 14:09:33.244674 [Capture MESSAGE] -- Capture started
 ** (wireshark:6996) 14:09:33.244770 [Capture MESSAGE] -- File: "/tmp/wiresha
rk_LoopbackZ4YLO3.pcapng"
 ** (wireshark:6996) 14:09:47.102265 [Capture MESSAGE] -- Capture Stop ...
 ** (wireshark:6996) 14:09:47.141731 [Capture MESSAGE] -- Capture stopped.
 ** (wireshark:6996) 14:09:53.212396 [Capture MESSAGE] -- Capture Start ...
 ** (wireshark:6996) 14:09:53.251408 [Capture MESSAGE] -- Capture started
 ** (wireshark:6996) 14:09:53.251462 [Capture MESSAGE] -- File: "/tmp/wiresha
rk_LoopbackQZGLO3.pcapng"
 ** (wireshark:6996) 14:12:02.084724 [Capture MESSAGE] -- Capture Stop ...
 ** (wireshark:6996) 14:12:02.126538 [Capture MESSAGE] -- Capture stopped.
 ** (wireshark:6996) 14:33:19.292091 [(none) WARNING] -- Error loading Implem
```

3. Attack Simulation

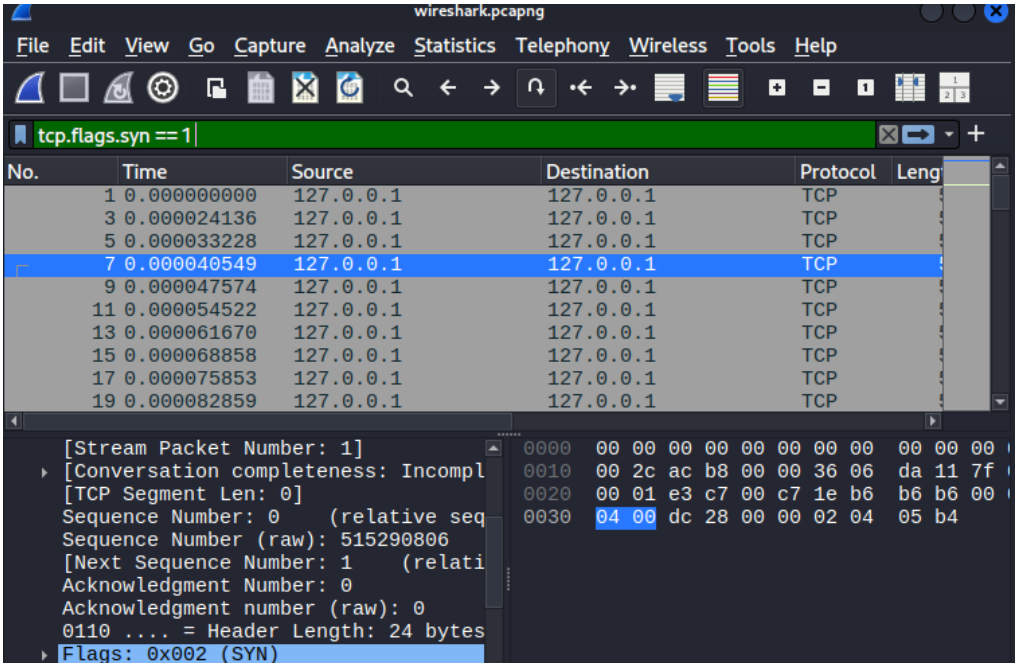
A port scanning attack was simulated using Nmap:



This command performs a **TCP SYN scan**, which sends SYN packets to multiple ports without completing the full TCP handshake.

4. Packet Filtering

To identify suspicious traffic, the following Wireshark filter was applied:
This filter isolates all SYN packets, which represent connection initiation attempts.



5. Automation using Python

To enhance analysis efficiency, a Python script was developed using the Scapy library.

Installation:

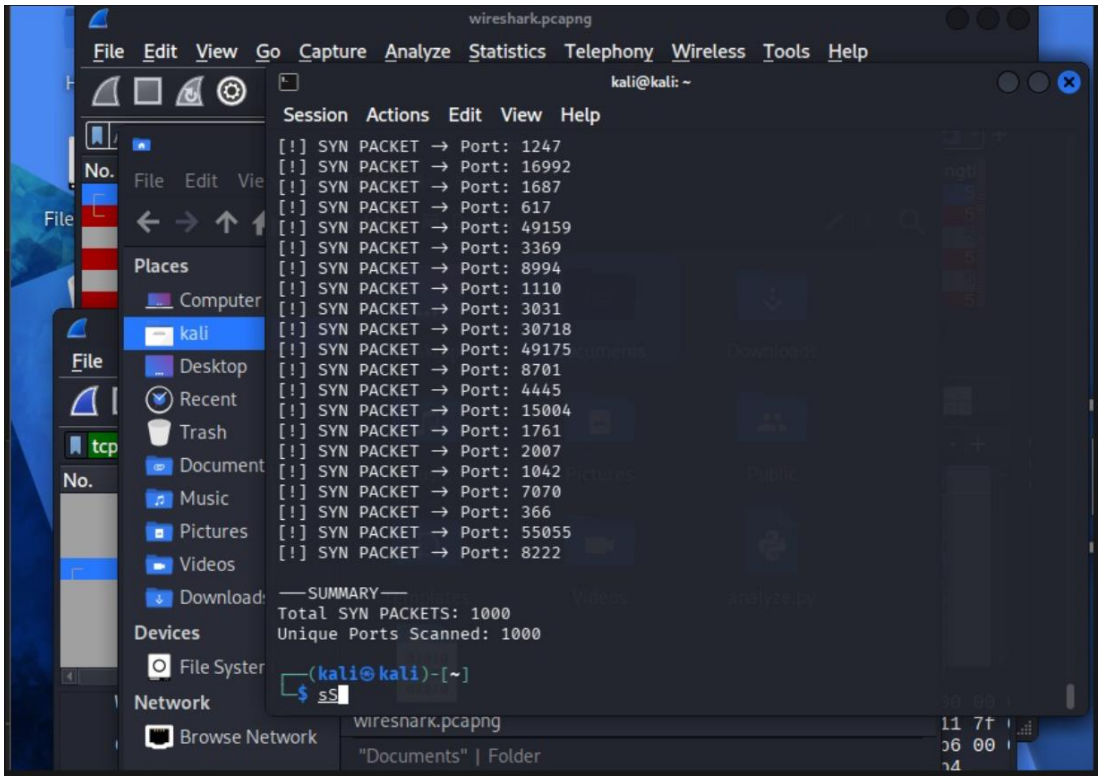
sudo apt update
sudo apt install python3-scapy -y

Script Execution:

- python3 analyze.py

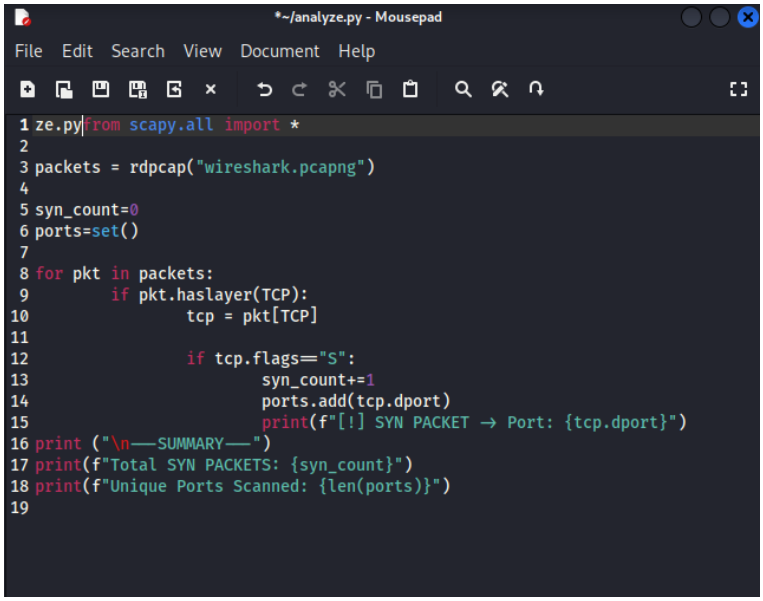
```
┌──(kali㉿kali)-[~]
└─$ python3 analyze.py
[!] SYN PACKET → Port: 587
[!] SYN PACKET → Port: 8888
[!] SYN PACKET → Port: 111
[!] SYN PACKET → Port: 199
[!] SYN PACKET → Port: 993
[!] SYN PACKET → Port: 53
[!] SYN PACKET → Port: 443
[!] SYN PACKET → Port: 110
[!] SYN PACKET → Port: 1025
[!] SYN PACKET → Port: 1720
[!] SYN PACKET → Port: 3389
[!] SYN PACKET → Port: 22
[!] SYN PACKET → Port: 554
[!] SYN PACKET → Port: 5900
```

The script processes the .pcapng file and extracts:



- Total number of SYN packets
- Unique destination ports scanned

Script:



Results and Analysis

The Wireshark capture revealed a high volume of TCP SYN packets generated during the Nmap scan. These packets were directed toward multiple destination ports within a short time interval.

The observed behavior indicates a **port scanning attack**, where an attacker probes multiple ports to identify open services.

Key observations:

- A large number of SYN packets were captured
- Each packet targeted a different port
- No complete TCP handshake was observed
- Traffic was concentrated within a short duration

The Python script further analyzed the captured data and produced the following results:

- **Total SYN Packets:** 1000
- **Unique Ports Scanned:** 1000

This confirms that the system was subjected to a systematic port scan across a wide range of ports.

TCP SYN packets are used to initiate communication between devices. In a normal scenario, a SYN packet is followed by a SYN-ACK response and a completed handshake.

However, in this experiment:

- Multiple SYN packets were sent
- Connections were not fully established
- Ports were probed sequentially

This pattern is characteristic of **reconnaissance attacks**, where attackers scan systems to discover vulnerabilities.

The use of automation significantly improved the efficiency of analysis by reducing manual inspection of packets and enabling quick extraction of relevant information.

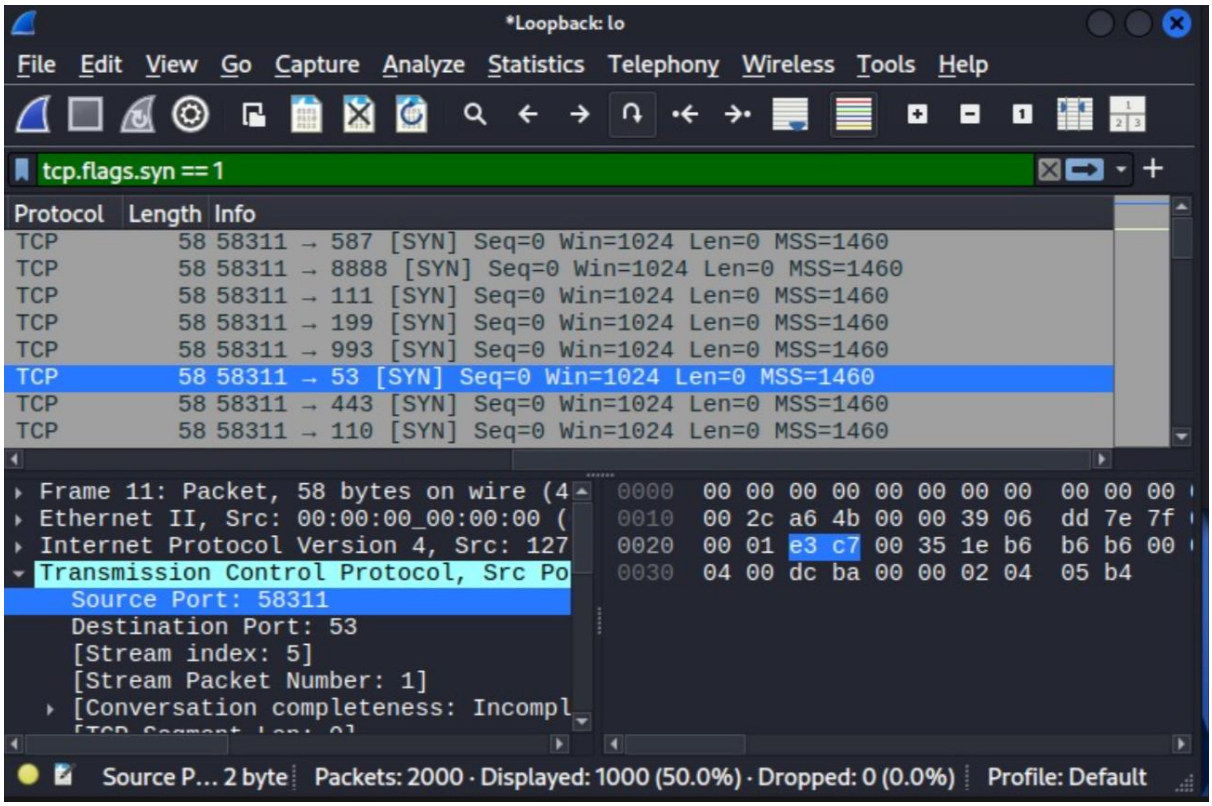
Conclusion

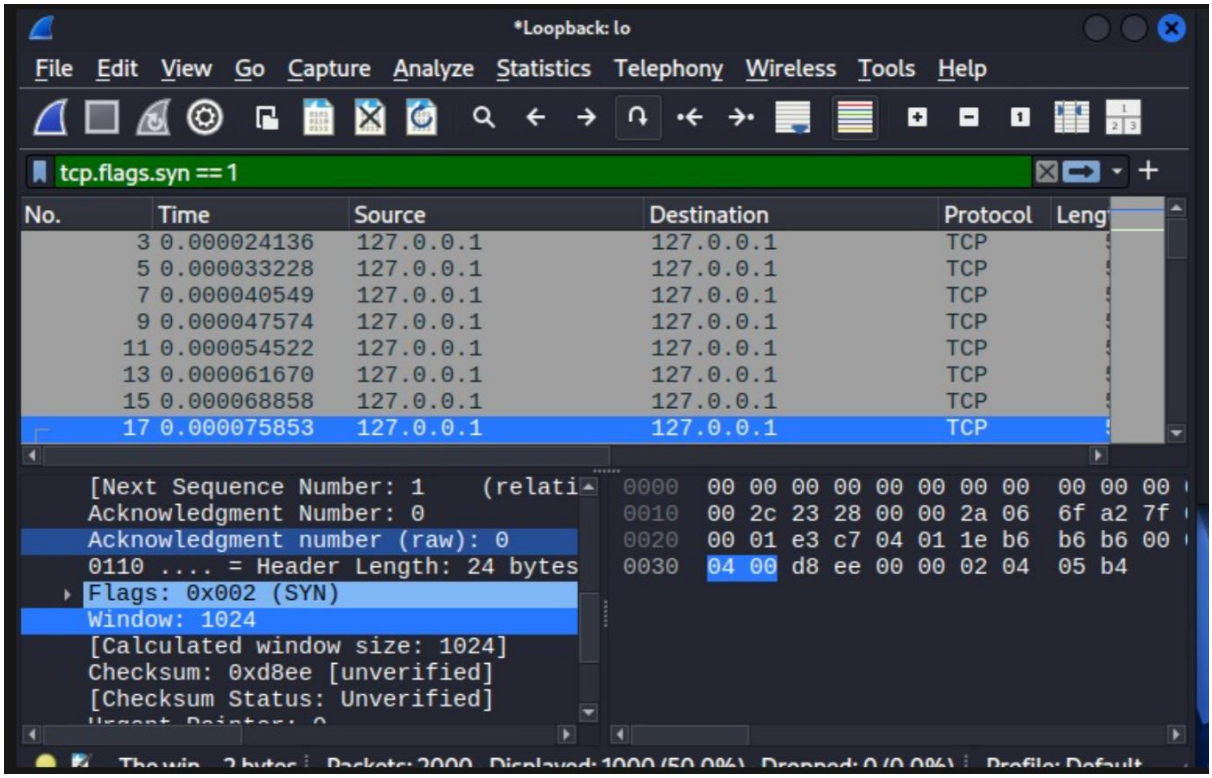
This experiment demonstrates the effectiveness of combining network monitoring tools and automation techniques for digital forensic analysis.

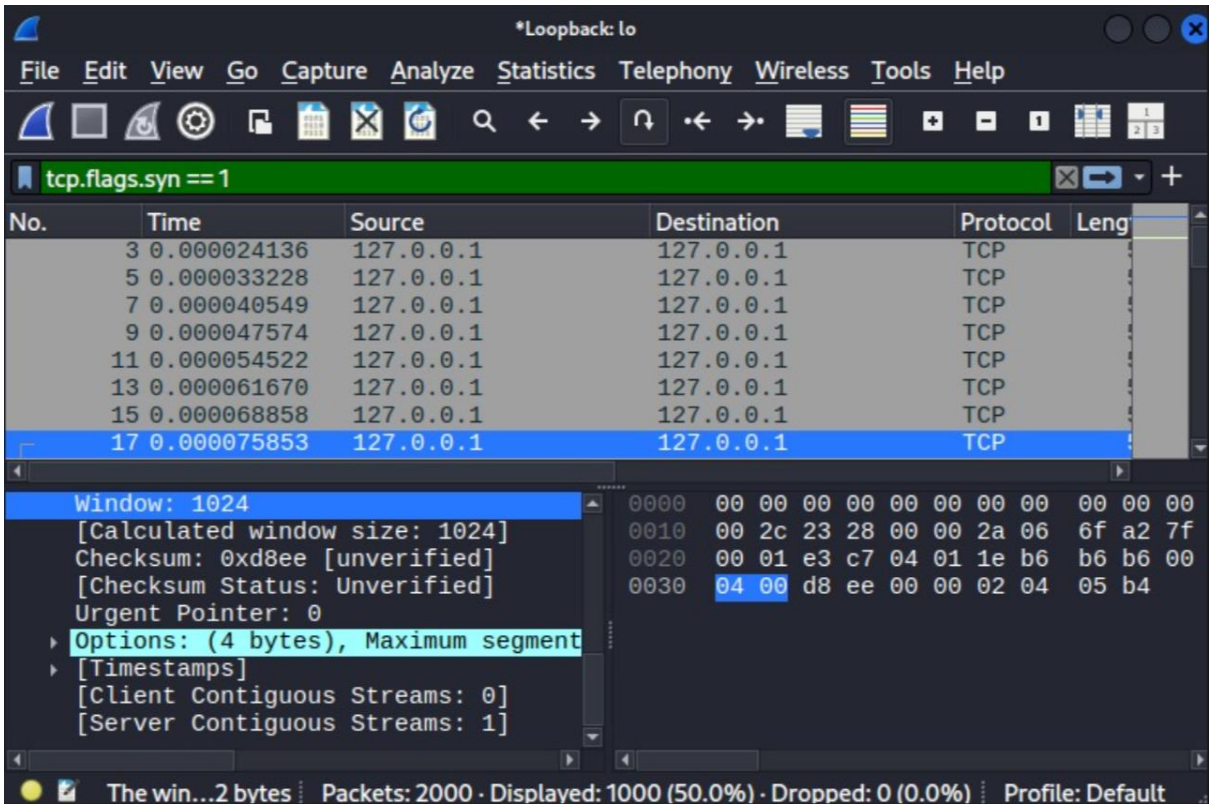
Wireshark successfully captured and identified suspicious network activity, while the Python script enabled efficient processing of large datasets. The results confirm that port scanning behavior can be detected through analysis of SYN packet patterns.

These techniques are essential for identifying potential cyber threats and improving cybersecurity incident response mechanisms.

TCP protocol details:




*Loopback: lo
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.flags.syn == 1
No. Time Source Destination Protocol Leng
3 0.000024136 127.0.0.1 127.0.0.1 TCP
5 0.000033228 127.0.0.1 127.0.0.1 TCP
7 0.000040549 127.0.0.1 127.0.0.1 TCP
9 0.000047574 127.0.0.1 127.0.0.1 TCP
11 0.000054522 127.0.0.1 127.0.0.1 TCP
13 0.000061670 127.0.0.1 127.0.0.1 TCP
15 0.000068858 127.0.0.1 127.0.0.1 TCP
17 0.000075853 127.0.0.1 127.0.0.1 TCP
[Next Sequence Number: 1 (relati
Acknowledgment Number: 0
Acknowledgment number (raw): 0
0110 .... = Header Length: 24 bytes
Flags: 0x002 (SYN)
Window: 1024
[Calculated window size: 1024]
Checksum: 0xd8ee [unverified]
[Checksum Status: Unverified]
0000 00 00 00 00 00 00 00 00 00 00 00
0010 00 2c 23 28 00 00 2a 06 6f a2 7f
0020 00 01 e3 c7 04 01 1e b6 b6 b6 00
0030 04 00 d8 ee 00 00 02 04 05 b4


*Loopback: lo
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.flags.syn == 1
No. Time Source Destination Protocol Leng
3 0.000024136 127.0.0.1 127.0.0.1 TCP
5 0.000033228 127.0.0.1 127.0.0.1 TCP
7 0.000040549 127.0.0.1 127.0.0.1 TCP
9 0.000047574 127.0.0.1 127.0.0.1 TCP
11 0.000054522 127.0.0.1 127.0.0.1 TCP
13 0.000061670 127.0.0.1 127.0.0.1 TCP
15 0.000068858 127.0.0.1 127.0.0.1 TCP
17 0.000075853 127.0.0.1 127.0.0.1 TCP
Window: 1024
[Calculated window size: 1024]
Checksum: 0xd8ee [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (4 bytes), Maximum segment
[Timestamps]
[Client Contiguous Streams: 0]
[Server Contiguous Streams: 1]
0000 00 00 00 00 00 00 00 00 00 00 00
0010 00 2c 23 28 00 00 2a 06 6f a2 7f
0020 00 01 e3 c7 04 01 1e b6 b6 b6 00
0030 04 00 d8 ee 00 00 02 04 05 b4
The win...2 bytes Packets: 2000 · Displayed: 1000 (50.0%) · Dropped: 0 (0.0%) Profile: Default